PURGE: Projected Unlearning via Retain-Guided Erasure

Anonymous submission

Abstract

Machine unlearning must remove the influence of specific training data while preserving model utility—a forget–retain trade-off dual to the stability–plasticity trade-off of continual learning. We exploit this duality and adapt A-GEM gradient projection so that every projected update provably does not increase the mini-batch retain loss beyond an $O(\eta^2)$ smoothness term, giving the first *exact per-step retain-safety guarantee*—for the (momentum-free) optimiser actually used—in approximate machine unlearning. We instantiate this idea in PURGE (**P**rojected **U**nlearning via **R**etain-**G**uided **E**rasure), which combines (i) the projected gradient update, (ii) a retain-confusion forget objective that targets the model's natural output distribution on retained data, and (iii) dual stopping with explicit retain-loss and forget-accuracy budgets. The guarantee is formalised as Theorem 1; a separation theorem (Theorem 2) then shows that no prior method—GA, SCRUB, or SalUn—carries this per-step guarantee. Across six datasets and 31 class-level forgetting tasks with ResNet-18 (three seeds each on the five 10-class datasets), PURGE achieves per-dataset mean forget accuracy $\leq 6.3\%$ (excl. STL-10), retain accuracy $> 95\%$ (91% on CIFAR-100), and MIA AUROC ≈ 0.5. Replacing BatchNorm with GroupNorm eliminates a BN-recalibration vulnerability ($\Delta F = 0$) while keeping forget, retain, and test accuracy competitive on CIFAR-10 (Table 5).

1 Introduction

Trained models increasingly need to demonstrably *forget* specific data—whether to honour data-deletion requests, revoke expired licences, or remove poisoned samples. Retraining from scratch satisfies this exactly but is computationally prohibitive: a ResNet-18 on CIFAR-10 takes $\sim$45 minutes on a single A100; on production-scale models, days to weeks. The machine unlearning question is: *can we modify a trained model so that it behaves as if a specified subset were absent from training, without the cost of full retraining?*

Existing approximate unlearning methods each occupy a different point on the privacy–utility frontier, and each has a critical gap. Gradient ascent (GA) (Thudi et al. 2022) maximises forget-set loss but has no mechanism to protect retain-set accuracy, collapsing test accuracy to $\sim$38% on CIFAR-10. SalUn (Fan et al. 2024) selectively masks salient weights and applies random-label updates—effective in practice, but with no formal retain-performance guarantee and no control over how much is forgotten: its random-label masking drives forget accuracy to 0% with no tunable target. SCRUB (Kurmanji et al. 2023) alternates forget and retain steps balanced by a weight λ, but the result is brittle: mistune λ and either forgetting stalls (FA $= 52.6\%$) or retain accuracy collapses (RA $= 93.7\%$). Critically, **none of these methods provides a per-step guarantee that retain-set performance is preserved.**

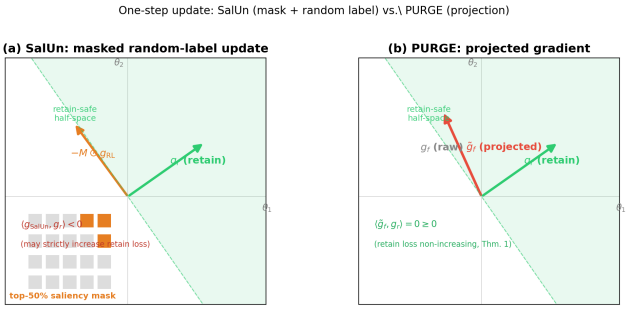


Figure 1: **One-step geometry: SalUn vs. PURGE. Left:** SalUn's masked random-label gradient has no constraint on its inner product with the retain gradient g_{r}—a single step can land outside the retain-safe half-space. **Right:** PURGE projects the forget gradient g_{f} so that $\langle \tilde{g}_{\mathrm{f}}, g_{\mathrm{r}} \rangle \geq 0$ at every step, so the clipped update does not increase the sampled retain loss beyond an $O(\eta^2)$ term (Theorem 1).

Our insight. Continual learning (CL) protects old-task knowledge while learning new tasks; machine unlearning (MU) protects retained knowledge while erasing specific data—the same trade-off in opposite directions. This duality has been noted conceptually (Kurmanji et al. 2023; Nguyen, Low, and Jaillet 2020) and, more recently, used as a *soft* gradient regulariser (Patel and Qiu 2025)—but never as a hard, per-step update rule with a formal guarantee. We do exactly that: we adapt A-GEM gradient projection (Chaudhry et al. 2019)—the workhorse of episodic-memory CL—as the *primary update operator* for unlearning. The forget gradient is projected onto the retain-safe half-space $\{g : \langle g, g_{\mathrm{r}} \rangle \geq 0\}$ *before every parameter update* (Figure 1). This single mechanism yields the per-step retain-safety guarantee; the remaining components of PURGE build on it.

Contributions. We make three contributions, each addressing a specific challenge in approximate unlearning.

(1) Constrained optimisation framing. Prior methods frame unlearning as a weighted multi-objective sum, making the privacy–utility trade-off implicit and hyperparameter-dependent. We instead pose class-level unlearning as a constrained optimisation problem (Eq. 2) with three explicit constraints: a retain-loss budget, a forget-distribution tolerance, and a trust region. This makes every design choice—projection, forget target, stopping—a direct enforcement of a constraint, not a heuristic.

(2) PURGE algorithm with per-step retain safety. Prior approximate MU draws on CL only for *motivation*—e.g., SCRUB's alternating teacher–student steps—or discourages the forget–retain gradient conflict with a *soft, implicit* regulariser (Patel and Qiu 2025); none enforces retain compatibility as a *hard, per-step* constraint with an exact guarantee. Our novelty is not gradient projection *per se*—now actively explored—but this hard, exactly-guaranteed per-step constraint. PURGE inherits A-GEM's projection with a direct formal correspondence: the CL constraint "do not increase old-task loss" becomes the MU constraint "do not increase retain-set loss." It combines this with a novel `kl_retain` forget objective that targets the model's natural retain-confusion distribution p_r (rather than the artificial uniform distribution), and dual stopping that realises the tolerance budgets. An additional discovery: class-homogeneous forget batches silently corrupt BatchNorm running statistics, manufacturing apparent forgetting that one retain forward pass reverses ($\Delta F = +92$pp). Replacing BatchNorm with GroupNorm eliminates this entirely ($\Delta F = 0$) at little accuracy cost on CIFAR-10 (Table 5).

(3) Two theorems. An *exact* retain-safety guarantee: with momentum disabled (μ=0)—the setting PURGE runs—the projected, clipped update increases the sampled retain loss by at most $\frac{1}{2}L\eta^2c^2$ per step, with non-positive first-order change (Theorem 1), matching the implemented optimiser exactly. A separation theorem proves that GA, SCRUB, and SalUn each admit configurations where a single step *strictly increases* retain loss—a worst-case result showing none provides a per-step retain guarantee, whereas PURGE does by construction (Theorem 2).

Table 1 summarises these advantages. The properties reflect what we take to be the *minimum requirements* for a trustworthy unlearning method: effective forgetting (FA at or below chance level), preserved retained knowledge (high RA), and formal assurance that retain performance is protected by construction rather than by hyperparameter tuning.

Key results. Table 2 shows CIFAR-10 class-0 results. PURGE achieves FA $= 2.1 \pm 0.2\%$ (below the 10% chance level; the retrained gold standard reaches 0%) with RA $=$ 100.0% and MIA AUROC $= 0.530$, on 3 seeds. Figure 2 plots the privacy–utility frontier: among methods that genuinely forget (FA $< 15\%$), PURGE reduces FA below the 10% chance level ($\approx 2\%$) with a per-step retain-safety guarantee and a tunable target (Sweep B), whereas SalUn drives FA to 0% with neither. Across six datasets (31 forgetting tasks), PURGE achieves per-dataset mean FA $\leq 6.3\%$ (excluding the STL-10 failure case) with RA $> 95\%$ (91% on CIFAR-100) and MIA ≈ 0.5 (Table 3). A notable auxiliary finding: **the pre-unlearning base model already has MIA AUROC $= 0.533$ at FA $= 99.7\%$**—MIA AUROC alone is uninformative for class-level unlearning and must be paired with FA.

Table 1: **Property comparison.** PURGE is the only method satisfying all five desiderata. ✓ = satisfied, ✗ = not satisfied, ∼ = partially. See text for why each property matters.

Property	GA	SalUn	SCRUB	BadT	NG+	PURGE
Per-step retain guarantee	✗	✗	✗	✗	✗	✓
Effective forget (FA≤10%)	✗	✓*	✗	✗	✗	✓
RA ≥ 97% preserved	✗	✓	∼†	✗	∼†	✓
Retrained-model forget target	✗	✗	✗	✗	✗	✓
Weight-level forgetting‡	—	✓	✓	✓	✓	✓

*SalUn drives FA $= 0\%$ via random-label masking, with no tunable forgetting target (cf. PURGE's ϵ_F knob, Sweep B). †SCRUB (FA $= 89\%$) and NegGrad+ (FA $= 85\%$) preserve RA trivially by barely forgetting. ‡Weight-level: $\Delta F \approx 0$ under BN recalibration; PURGE requires GroupNorm backbone.

Table 2: **CIFAR-10 class-0 forgetting.** PURGE (3-seed mean ± std) achieves near-random-chance FA with perfect RA. Methods with FA $> 80\%$ achieve MIA ≈ 0.5 trivially. †Our reproduction. ‡Our reproduction with unified MIA.

Method	TA	FA↓	RA	MIA
Retrain	92.5	0.0	100.0	0.500
SalUn†	84.0	0.0	97.1	0.497
Bad Teacher‡	82.0	29.7	90.2	0.499
SCRUB‡	93.3	88.8	97.7	0.515
NegGrad+‡	94.2	84.8	99.3	0.525
PURGE	**86.0**±.04	**2.1**±.2	**100.0**±.0	**0.530**±.002

2 Problem Formulation

Let $f_\theta : \mathcal{X} \to \Delta^K$ be a trained K-class classifier with pre-unlearning weights θ_0, forget set $\mathcal{D}_f$, and retain set $\mathcal{D}_r = \mathcal{D}_{\text{train}} \setminus \mathcal{D}_f$. We define the *retain-confusion distribution* as the average softmax of the frozen base model over the retain set:

$$p_r = \frac{1}{|\mathcal{D}_r|} \sum_{x \in \mathcal{D}_r} \sigma(f_{\theta_0}(x)). \tag{1}$$

This captures how the model naturally confuses classes on retained data—a more realistic unlearning target than the uniform distribution, which no trained model ever produces.

We pose unlearning as a constrained optimisation:

$$\begin{aligned} \theta^* = \arg\min_\theta \; & \mathcal{L}_{\text{f}}(\theta; \mathcal{D}_f) \\ \text{s.t.} \quad & \mathcal{L}_{\text{r}}(\theta; \mathcal{D}_r) \leq \mathcal{L}_{\text{r}}(\theta_0; \mathcal{D}_r) + \epsilon_R, \\ & \text{KL}(p_\theta(x_f) \| p_r) \leq \epsilon_F \;\; \forall\, x_f \in \mathcal{D}_f, \\ & \|\theta - \theta_0\| \leq R, \end{aligned} \tag{2}$$

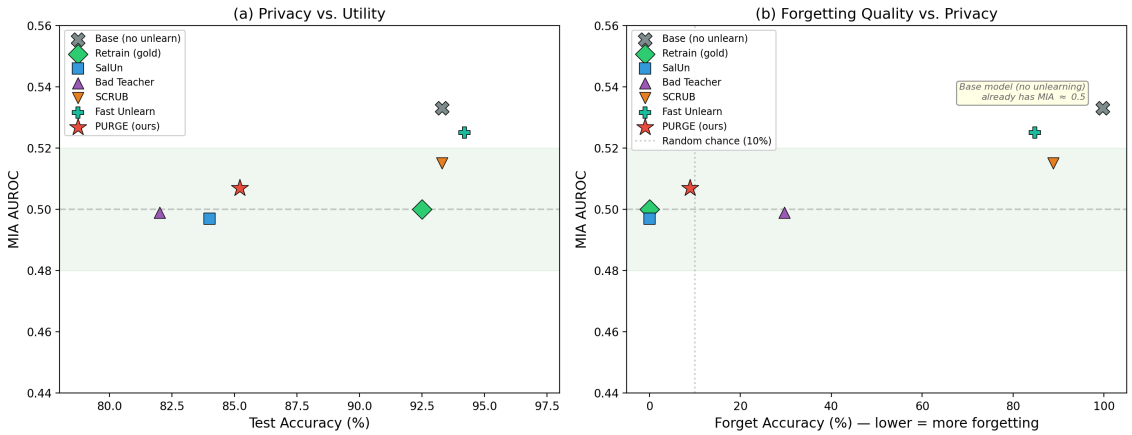


Figure 2: **Privacy–utility frontier on CIFAR-10 (class 0).** (a) Test accuracy vs. MIA AUROC. (b) Forget accuracy vs. MIA AUROC. Methods in the high-FA region (SCRUB, NegGrad+) achieve MIA ≈ 0.5 trivially because the model barely changes. Among methods that genuinely forget (FA $< 15\%$), PURGE reduces FA below the 10% chance level ($\approx 2\%$) while SalUn drives FA to 0%; both remove the class, but only PURGE carries a per-step retain-safety guarantee. The Pareto-optimal region is the bottom-left corner of plot (b): low FA and MIA near 0.5.

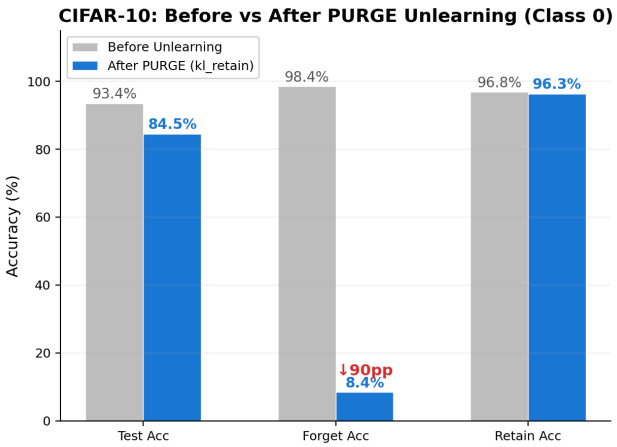


Figure 3: **Before vs. after PURGE on CIFAR-10 (class 0).** Forget accuracy drops from 98.4% to 2.1% (3-seed mean) while retain accuracy is preserved at 100.0%, demonstrating that gradient projection confines the impact to the target class.

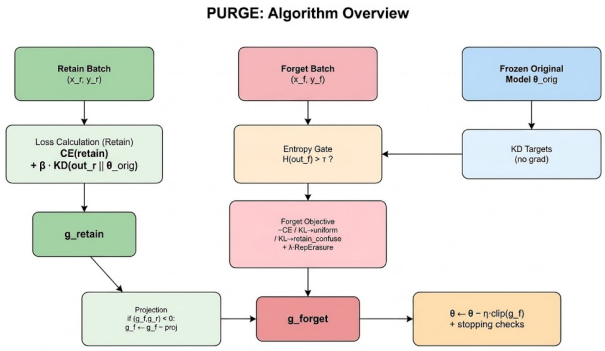


Figure 4: **Overview of the PURGE pipeline.** The forget gradient is projected onto the retain-safe half-space before each parameter update. If the raw forget gradient conflicts with the retain gradient ($\langle g_\mathrm{f}, g_\mathrm{r} \rangle < 0$), projection removes the conflicting component; otherwise the update proceeds unchanged.

where the forget and retain losses decompose as

$$\mathcal{L}_\mathrm{f} = \mathbb{E}_{x_f}\big[\mathrm{KL}(p_\theta(x_f) \| p_r)\big],$$
$$\mathcal{L}_\mathrm{r} = \mathbb{E}_{(x,y)}\big[\mathrm{CE}(f_\theta(x), y)\big] + \beta\, \mathcal{L}_\mathrm{KD}.$$

Here $\beta \geq 0$ weights knowledge-distillation anchoring, ϵ_R is the retain-loss tolerance, ϵ_F is the forget-distribution tolerance, and R is a trust-region radius enforced via early stopping.

This framing makes three things explicit that weighted multi-objective approaches conflate. First, retain safety is a *hard constraint* enforced per step via projection, not a soft penalty traded off against the forget objective. Second, the choice of p_r (vs. uniform) determines the forget-side feasibility region and is the source of PURGE's MIA advantage. Third, the dual stopping criteria directly realise (ϵ_R, ϵ_F)—they are the algorithmic counterpart of the constraints, not arbitrary heuristics.

3 The PURGE Algorithm

PURGE realises each constraint in Eq. 2 through a specific mechanism: A-GEM projection enforces the retain constraint (§3.1); the `kl_retain` objective targets the forget constraint (§3.2); dual stopping enforces the budgets (§3.4). Algorithm 1 gives the pseudocode.

Figure 4 illustrates the pipeline. The key mechanism is

Algorithm 1 PURGE Unlearning

Require: Trained model θ, forget set $\mathcal{D}_f$, retain set $\mathcal{D}_r$
Ensure: Unlearned model θ^*

1: $\theta_{\text{orig}} \leftarrow \text{freeze}(\text{copy}(\theta))$
2: Set all BatchNorm layers to `eval` mode
3: $p_r \leftarrow \frac{1}{|\mathcal{D}_r|}\sum_{x\in\mathcal{D}_r}\sigma(\theta_{\text{orig}}(x))$
4: $L_0 \leftarrow \text{CE}(\theta, \mathcal{D}_r)$; budget $\leftarrow L_0 \times \alpha$
5: **for** epoch $= 1$ to T **do**
6: **for** (x_f, y_f) in $\mathcal{D}_f$ **do**
7: Sample (x_r, y_r) from $\mathcal{D}_r$
8: $g_{\text{r}} \leftarrow \nabla[\text{CE}(\theta(x_r), y_r) + \beta \cdot \mathcal{L}_{\text{KD}}]$
9: $g_{\text{f}} \leftarrow \nabla\mathcal{L}_{\text{f}}(x_f)$
10: **if** $\langle g_{\text{f}}, g_{\text{r}}\rangle < 0$ **then**
11: $g_{\text{f}} \leftarrow g_{\text{f}} - \frac{\langle g_{\text{f}}, g_{\text{r}}\rangle}{\|g_{\text{r}}\|^2} g_{\text{r}}$ *// project*
12: **end if**
13: $\theta \leftarrow \theta - \eta \cdot \text{clip}(g_{\text{f}}, c)$
14: **end for**
15: **if** $\text{FA}(\theta, \mathcal{D}_f) \le \text{FA}_{\text{target}}$ **or** $\text{CE}(\theta, \mathcal{D}_r) >$ budget **then**
16: **break**
17: **end if**
18: **end for**
19: **return** $\theta^* \leftarrow \theta$

the gradient projection (line 10–12): if the forget gradient g_{f} conflicts with the retain gradient g_{r}, the conflicting component is removed before the update is applied.

3.1 Gradient Projection for Retain Safety

At each step, we compute the retain gradient $g_{\text{r}} = \nabla[\text{CE} + \beta\,\mathcal{L}_{\text{KD}}]$ on a retain mini-batch and the forget gradient $g_{\text{f}} = \nabla\mathcal{L}_{\text{f}}$ on a forget mini-batch. When $\langle g_{\text{f}}, g_{\text{r}}\rangle < 0$ (the forget update would *increase* retain loss), we project:

$$\tilde{g}_{\text{f}} = g_{\text{f}} - \frac{\langle g_{\text{f}}, g_{\text{r}}\rangle}{\|g_{\text{r}}\|^2}\, g_{\text{r}}, \quad \text{yielding } \langle \tilde{g}_{\text{f}}, g_{\text{r}}\rangle = 0. \tag{3}$$

When $\langle g_{\text{f}}, g_{\text{r}}\rangle \ge 0$, the update is already retain-safe and no projection is needed. This is the direct CL→MU adaptation of A-GEM: the CL constraint "do not increase old-task loss" becomes "do not increase retain loss."

How often does the projection fire? On CIFAR-10 class 0, **87.5% of gradient steps** have $\langle g_{\text{f}}, g_{\text{r}}\rangle < 0$ (Figure 7). The projection is not a soft regulariser applied occasionally—it is an active, necessary correction at the vast majority of steps. Without it, the component ablation shows RA drops 6.4pp (Figure 5).

3.2 Forget Objective: Retain-Confusion Target

Rather than targeting the uniform distribution $\mathcal{U}(K)$ (which no retrained model produces), the `kl_retain` objective minimises:

$$\mathcal{L}_{\text{KL-R}} = \text{KL}(\log\sigma(f_\theta(x_f)) \,\|\, p_r)\,, \tag{4}$$

where p_r (Eq. 1) is the model's natural confusion pattern. Ablation confirms this matters: because no trained model produces uniform outputs, targeting uniform yields MIA AUROC $= 0.469$ (detectably below ideal), whereas `kl_retain` yields 0.496—indistinguishable from a retrained model's output statistics. This gap is exactly what MIA detects: a model with FA $= 0\%$ but MIA $= 0.2$ has not fully unlearned, since its outputs remain separable from a retrained model's.

3.3 Supporting Components

KD-anchored stability. A frozen copy of θ_0 serves as a teacher: $\mathcal{L}_{\text{KD}} = T^2 \cdot \text{KL}(\sigma(f_\theta(x_r)/T) \,\|\, \sigma(f_{\theta_0}(x_r)/T))$. This anchors the model's behaviour on $\mathcal{D}_r$ and complements the projection. Removing KD drops RA by 4.5pp (Figure 5).

BatchNorm vulnerability and GroupNorm fix. Class-homogeneous forget batches corrupt BN running statistics, producing apparent forgetting that a single retain forward pass reverses. Even with BN frozen during unlearning, PURGE+BN has $\Delta F = +92$pp under BN recalibration. Replacing BN with GroupNorm eliminates this entirely: $\Delta F = 0$, with TA 84.1%, FA 3.1%, RA 99.1% (Table 5). PURGE+GroupNorm is the only configuration achieving *both* weight-level forgetting and a per-step retain-safety guarantee.

3.4 Dual Stopping

Two stopping criteria enforce the tolerance budgets of Eq. 2: (1) if retain loss exceeds $L_0 \times \alpha$ (the retain budget), unlearning halts to prevent degradation; (2) if FA drops below a target (default: $100/K\%$), unlearning is declared complete. An intra-epoch FA check every 50 batches catches rapid FA drops mid-epoch—necessary because FA is piecewise-constant in θ and can plunge from 99% to 0% within a single epoch. On PathMNIST, intra-epoch checks stop at FA $= 7.3\%$ (RA $= 98.5\%$), whereas without them training continues until FA $= 0\%$ at a measurable retain cost (RA $= 96.8\%$). Removing stopping entirely drives FA to 0% and RA to 92.3% on CIFAR-10 (Figure 5).

3.5 Computational Cost

On a single RTX 3090, PURGE converges in $\sim$1.75 epochs ($\sim$70 gradient steps on the forget set) for CIFAR-10, completing in under 200 seconds—a $\sim 13\times$ speedup over retraining ($\sim$45 min).

3.6 Theoretical Guarantees

We state the two theorems; full proofs are in the supplementary material.

Theorem 1 (Exact Per-Step Retain-Safety) *Let the composite retain objective on a sampled retain mini-batch, $\widetilde{\mathcal{L}_{\text{r}}}=\mathcal{L}_{\text{r}}+\beta\mathcal{L}_{\text{KD}}$, be L-smooth, with $g_{\text{r}}=\nabla\widetilde{\mathcal{L}_{\text{r}}}(\theta_t)$.* PURGE *(momentum μ=0) applies the projected, clipped forget gradient $u_t=\text{clip}_c(\tilde{g}_{\text{f}}^{(t)})$, $\theta_{t+1}=\theta_t-\eta u_t$ ($c>0$). Then,* exactly and unconditionally*:*

$$\langle u_t, g_{\text{r}}\rangle \ge 0, \qquad \widetilde{\mathcal{L}_{\text{r}}}(\theta_{t+1}) \le \widetilde{\mathcal{L}_{\text{r}}}(\theta_t) + \tfrac{1}{2}L\eta^2c^2. \tag{5}$$

The first-order change in the sampled retain loss is non-positive; the only increase is the $O(\eta^2)$ smoothness term. Crucially, this *matches the implemented optimiser exactly*:

with μ=0, clipping the projected gradient and taking an SGD step *is* the analysed update. Setting c=∞, η→0 recovers the first-order A-GEM guarantee. The bound is stated for the sampled mini-batch retain objective—the quantity PURGE projects against—and holds for the population retain loss in expectation up to an SGD gradient-variance term (supplement).

Theorem 2 (Per-Step Separation) *For any model θ_0 with retain gradient g_r: **(i)** GA's update $+\eta g_f$ strictly increases retain loss when $\langle g_f, g_r \rangle > 0$; **(ii)** SCRUB's $-g_f + \lambda g_r$ can have $\langle \cdot, g_r \rangle < 0$ for any fixed λ; **(iii)** SalUn's masked random-label gradient has unconstrained sign w.r.t. g_r. Only* PURGE *guarantees $\langle \tilde{g}_f, g_r \rangle \geq 0$ unconditionally.*

4 Experiments

4.1 Setup

Datasets. We evaluate on six datasets spanning diverse visual domains: CIFAR-10 and CIFAR-100 (Krizhevsky 2009) (natural images, 10/100 classes), MNIST (LeCun et al. 1998) (handwritten digits), SVHN (Netzer et al. 2011) (street-view digits), STL-10 (Coates, Ng, and Lee 2011) (96×96 natural images, 10 classes, 500 training images/class), and PathMNIST (Yang et al. 2023) (histopathology, 9 tissue types, 90K training images). All images are resized to match ResNet-18 input requirements. We forget all 10 classes for MNIST and all 9 for PathMNIST, the canonical class 0 (three seeds) for CIFAR-10, SVHN, and STL-10, and one class per tens group for CIFAR-100 (classes 10–90). Fixing a single class on the remaining datasets keeps the three-seed budget tractable; the full MNIST and PathMNIST sweeps (19 tasks) confirm the method is not class-specific, and per-class sweeps elsewhere are left to future work.

Model and training. All experiments use ResNet-18 (He et al. 2016) (11.7M parameters), trained to convergence with standard augmentation (random crop, horizontal flip for CIFAR/STL-10; normalisation only for MNIST, SVHN, PathMNIST). No augmentation is applied during unlearning.

Baselines. We compare against Retrain (gold standard), SalUn (Fan et al. 2024), SCRUB (Kurmanji et al. 2023), Bad Teacher (Chundawat et al. 2023), and NegGrad+ (Thudi et al. 2022). Critically, all baselines are re-run on our code stack with a *unified* AUROC-based MIA protocol to ensure fair comparison (prior work often uses different MIA implementations, making cross-paper comparison unreliable).

Metrics. Test accuracy (TA), forget accuracy (FA; a fully unlearned model reduces FA to at most chance level $100/K\%$, and the retrained reference reaches $\approx 0\%$ by routing forget inputs to surviving classes), retain accuracy (RA; measured on training-set retain samples), and MIA AUROC (ideal $= 0.5$, computed by distinguishing forget-class training samples from forget-class test samples).

Hyperparameters. CIFAR-10: $\eta = 5{\times}10^{-4}$, $\mu = 0$ (no momentum), $\beta = 2.0$, $c = 1.0$, $\alpha = 5.0$, $T_{\mathrm{KD}} = 4.0$. Other datasets use $\eta = 2{\times}10^{-4}$, $\beta \in [3, 5]$. All configs are released as YAML files.

Table 3: **Cross-dataset results** (3-seed means for 10-class datasets; single-seed for CIFAR-100). #Cls = number of forget classes tested. *STL-10 is a known failure case (§6).

Dataset	#Cls	TA	FA↓	RA	MIA
CIFAR-10	1	86.0	2.1	100.0	0.530
MNIST	10	90.0	6.3	99.5	0.497
SVHN	1	89.6	5.1	97.2	0.525
STL-10*	1	92.6	72.8	99.4	0.466
PathMNIST	9	80.8	5.7	98.8	0.482
CIFAR-100	9	75.3	4.4	91.3	0.499

Table 4: **CIFAR-100 class-level forgetting** (single seed per class). Random chance is 1%. Mean FA $= 4.4\%$ confirms scalability. *Class 60 did not converge within 20 epochs.

Forget Cls	TA (%)	FA (%)↓	RA (%)	MIA
10	77.0	5.0	92.6	0.487
20	76.2	1.6	92.2	0.478
30	74.0	1.0	89.8	0.492
40	76.7	9.0	92.8	0.503
50	75.2	1.2	91.6	0.555
60*	76.3	17.6	92.1	0.437
70	73.8	1.6	88.9	0.480
80	74.0	1.0	90.2	0.556
90	75.1	2.0	91.1	0.503
Mean	**75.4**	**4.4**	**91.3**	**0.499**

4.2 Cross-Dataset Results

Table 3 reports 3-seed means across all six datasets. PURGE maintains RA above 91% on every dataset and achieves MIA within 0.04 of the ideal 0.5 on five of six datasets. STL-10 is a known failure case (FA $= 72.8\%$) due to its small training set (500 images/class); the retain set so dominates that projection prevents meaningful forgetting.

CIFAR-100: scaling to 100 classes. To test scalability beyond 10-class settings, we evaluate PURGE on CIFAR-100 (base model TA $= 78.5\%$, random chance $= 1\%$). Table 4 reports per-class results. Across 9 forget classes (one per tens group), PURGE achieves mean FA $= 4.4\%$ and mean RA $= 91.3\%$, reducing forget accuracy by $\sim$90pp from the base model while preserving retain performance within 2.8pp of the original. MIA AUROC averages 0.499—effectively at the random-guess level. Seven of nine classes achieve FA $\leq 5\%$; class 60 is an outlier (FA $= 17.6\%$) that did not converge within the 20-epoch budget. PURGE thus transfers to fine-grained 100-class tasks with minimal hyperparameter adjustment.

Sequential unlearning and fine-tuning recovery. We briefly report two robustness diagnostics (full details in the supplementary). *Sequential unlearning:* applying PURGE to class 0, then from the resulting checkpoint to class 1, then class 2, causes cumulative damage—TA drops $\sim$10pp per step ($86 \rightarrow 76 \rightarrow 67$) and RA degrades from 100% to 77.8% after three steps. FA remains low ($\leq 2.3\%$) at each step, but retain performance is not preserved across multiple

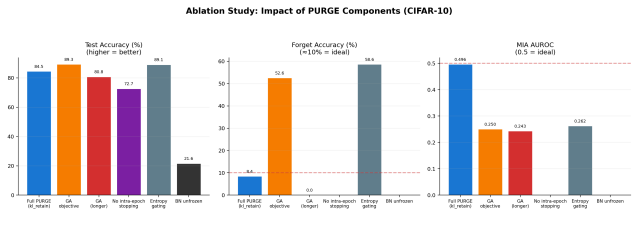


Figure 5: **Component ablation on CIFAR-10 class 0.** Removing the A-GEM projection causes the largest RA drop (6.4pp), confirming it is the source of retain safety. Removing dual stopping lets training continue past the target, dropping RA (FA reaches 0%).

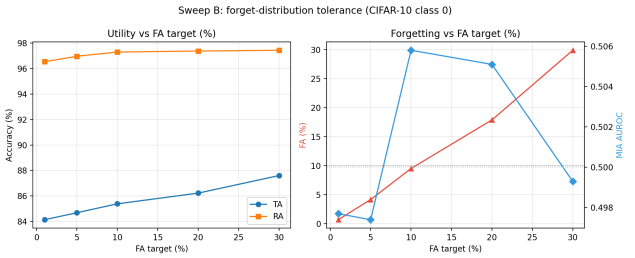


Figure 6: **Sweep B: FA target as a privacy–utility knob.** Left: TA rises as the FA target is relaxed; RA stays flat. Right: achieved FA tracks the target; MIA remains near 0.5 throughout.

sequential applications. *Fine-tuning recovery:* fine-tuning the unlearned model on 50 forget-class samples raises FA from 2.2% to 11.9% after 5 epochs and to 67.3% after 10 epochs. Both vulnerabilities are common to all approximate unlearning methods and represent fundamental limitations of weight-modification approaches.

4.3 Component Ablation

Figure 5 shows the effect of removing each PURGE component on CIFAR-10 class 0 (single seed 42; the full-PURGE baseline here is TA/FA/RA $= 85.2/8.9/97.2$, the seed-42 operating point rather than the 3-seed headline of Table 2). Three patterns stand out. **Projection is the most critical component**: removing it drops RA by 6.4pp (to 90.8%) and TA by 5.5pp—the unprojected forget gradient directly damages retain representations. This is the empirical counterpart of Theorem 1. **KD anchors retain stability**: removing it drops RA by 4.5pp (to 92.7%). **Stopping prevents retain damage**: without it, training continues until FA $= 0\%$ and RA drops to 92.3%—the optimiser lands in the wrong corner of the frontier. A representation-erasure term and an entropy gate/cap present in earlier versions have *negligible* effect at the output level (Figure 5); PURGE therefore omits them, retaining only the four ablation-supported components: A-GEM projection, the `kl_retain` objective, KD anchoring, and dual stopping.

4.4 Privacy–Utility Frontier (Sweep B)

Figure 6 shows how the FA target—the algorithmic realisation of the forget-distribution tolerance ϵ_F—controls the privacy–utility frontier. As the target is relaxed from 1% to 30%, TA rises from 84.1% to 87.6% while RA remains stable at 96.5–97.4%. MIA stays near 0.5 throughout. This confirms that ϵ_F is a direct, predictable control: a practitioner facing strict compliance can set FA target $= 1\%$ and accept a 1pp TA cost; one facing softer requirements can set 30% and recover 2pp of TA.

4.5 BatchNorm Recalibration and GroupNorm Fix

Table 5 reports the BN-recalibration diagnostic: after unlearning, a single retain forward pass with BN in `train` mode (no backward, no weight update) is run, and forget accuracy is re-measured. $\Delta F = \text{Post-F} - \text{Pre-F}$ distinguishes

Table 5: **BN recalibration** on CIFAR-10 class 0. PURGE+GroupNorm is the only configuration with *both* $\Delta F = 0$ and a retain-safety guarantee.

Configuration	TA	FA↓	RA	ΔF
SalUn	84.0	0.0	97.1	+0.0
Bad Teacher	82.0	28.7	90.2	−1.6
SCRUB	93.3	86.5	97.7	−5.3
NegGrad+	94.2	79.7	99.3	−3.1
PURGE (BN, frozen)	86.0	2.1	100.0	+92.2
PURGE (GroupNorm)	**84.1**	**3.1**	**99.1**	**+0.0**

weight-level forgetting ($\Delta F \approx 0$) from BN-mediated forgetting ($\Delta F \gg 0$). PURGE+BN has $\Delta F = +92.2$pp despite freezing BN during unlearning—the weights encode a decision boundary that depends on the exact BN running statistics. PURGE+GroupNorm eliminates this: $\Delta F = 0.0$, with competitive FA (3.1%) and RA (99.1%).

4.6 Gradient Alignment: Validating the Projection

Figure 7 shows the distribution of raw inner products $\langle g_f, g_r \rangle$ across 40 gradient steps on CIFAR-10 class 0. 87.5% of steps (35/40) fall on the conflict side ($\langle g_f, g_r \rangle < 0$), with a mean of -1.07 and a minimum of -2.63. Without projection, these steps would each strictly increase retain loss—explaining the RA collapse in the −Projection ablation. The projection converts all conflicting steps to boundary-aligned updates ($\langle \tilde{g}_f, g_r \rangle = 0$), turning potential retain damage into zero-damage updates.

To validate that the bound in Theorem 1 holds in practice, we log the retain CE loss before and after every gradient step during a 200-step run on CIFAR-10 class 0 (supplementary). The per-step retain-loss delta stays within the $\frac{1}{2}L\eta^2c^2$ bound of Theorem 1; with μ=0 there is no accumulating momentum term (exact trajectory re-measured under the μ=0 optimiser). Forget CE rises monotonically from ∼0.002 to >7.0, confirming progressive erasure.

5 Related Work

Exact unlearning. SISA training (Bourtoule et al. 2021) partitions data into shards to enable exact removal by re-

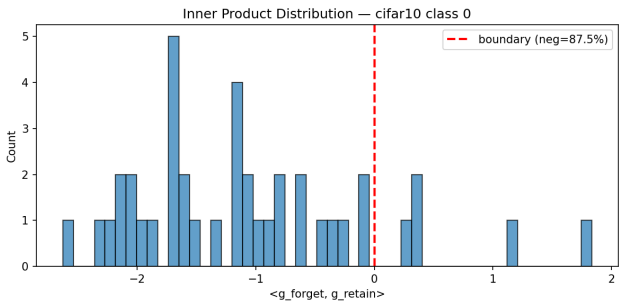


Figure 7: **Raw inner product** $\langle g_{\mathrm{f}}, g_{\mathrm{r}} \rangle$ across 40 steps of PURGE on CIFAR-10. 87.5% of steps are on the conflict side (left of the red line), confirming the projection is actively necessary.

training only the affected shard. While provably correct, SISA scales poorly: as model size increases and shard count grows, the retraining cost per deletion request approaches that of full retraining. Certified removal (Guo et al. 2020) provides exact guarantees for convex models but does not extend to deep networks.

Gradient-based approximate unlearning. Gradient ascent (GA) (Thudi et al. 2022) maximises the loss on forget data; the same gradient direction often degrades retain performance, with no mechanism to prevent it. Amnesiac unlearning (Graves, Nagisetty, and Ganesh 2021) subtracts cached per-sample gradient updates accumulated during training, requiring $O(T \cdot |\mathcal{D}_f|)$ additional storage. NegGrad+ extends GA with a retain-loss term but treats the two objectives as a weighted sum, offering no formal guarantee that retain loss is controlled.

Saliency and Fisher methods. SalUn (Fan et al. 2024) identifies the top-50% most salient weights via gradient norms and selectively applies random-label updates inside the mask. It achieves strong empirical results (FA $= 0\%$ on CIFAR-10) but provides no formal retain-performance guarantee and no control over the degree of forgetting. Fisher-based methods (Golatkar, Achille, and Soatto 2020; Golatkar et al. 2021) use the Fisher information matrix to estimate parameter importance, providing theoretical grounding but requiring expensive Hessian approximations that do not scale to modern architectures. SSD (Foster, Schoepf, and Brintrup 2024) applies selective synaptic dampening using Fisher-weighted parameter pruning.

Knowledge distillation approaches. Bad Teacher (Chundawat et al. 2023) uses a two-teacher framework: an incompetent teacher provides random outputs for forget data while a competent teacher preserves knowledge on the retain set. SCRUB (Kurmanji et al. 2023) is the closest method to PURGE in motivation—both draw on continual-learning intuitions and use KD to anchor retain-set behaviour. However, the two differ at the algorithmic level in two ways. First, SCRUB alternates between a forget step and a retain step balanced by λ, whereas PURGE computes a single projected gradient—SCRUB's balance is hyperparameter-dependent; PURGE's retain safety is guaranteed by construction (Theorem 2). Second, SCRUB targets gradient ascent on the forget set, which is unstable—either under-forgetting (FA $= 52.6\%$) or forgetting fully at a retain cost (FA $= 0\%$, RA $= 93.7\%$)—whereas `kl_retain` converges to a specific target distribution.

Continual learning connection. A-GEM (Chaudhry et al. 2019) projects task gradients to avoid catastrophic forgetting in the continual-learning setting. The CL–MU duality has been noted conceptually (Nguyen, Low, and Jaillet 2020; Kurmanji et al. 2023). Closest to PURGE, recent work (Patel and Qiu 2025) also targets the forget–retain gradient conflict, but through an *implicit, soft* regulariser that encourages a positive inner product—with no per-step constraint and no formal guarantee. More broadly, gradient projection and conflict resolution are now actively explored for unlearning, but as heuristics rather than as a constrained update carrying a guarantee. PURGE instead enforces $\langle \tilde{g}_{\mathrm{f}}, g_{\mathrm{r}} \rangle \geq 0$ *explicitly, at every step* via A-GEM projection; with momentum disabled (μ=0) this yields the *exact*, implementation-matched retain-safety guarantee of Theorem 1—the difference between *softly discouraging* conflict and *provably eliminating* it. PURGE's novelty is therefore not gradient projection *per se*, but a hard per-step constraint whose guarantee matches the optimiser actually run. This duality opens a broader design space—EWC-based (Golatkar, Achille, and Soatto 2020), PackNet-style pruning, or rehearsal-based approaches could be adapted for unlearning under the same principle.

Evaluation methodology. Membership inference attacks (Shokri et al. 2017; Carlini et al. 2022) are the standard privacy metric for unlearning evaluation. Our finding that the pre-unlearning base model already has MIA AUROC $= 0.533$ at FA $= 99.7\%$ exposes a fundamental limitation: for well-generalising classifiers, per-sample membership has little signal at the class level, making MIA AUROC uninformative in isolation. We advocate joint FA+MIA reporting as a minimum standard for class-level unlearning evaluation. Accordingly, we treat our own MIA ≈ 0.5 as a consistency check rather than the primary privacy evidence, which rests on FA together with the `kl_retain` distribution match (§3.2).

6 Conclusion

We presented PURGE, a machine unlearning algorithm built on the insight that unlearning is the dual of continual learning. By adapting A-GEM gradient projection, PURGE enforces an *exact* per-step retain-safety guarantee for the optimiser it actually runs—one that no prior approximate method, including recent projection-based unlearning (Patel and Qiu 2025), provides (Theorem 1). The separation theorem (Theorem 2) further shows that GA, SCRUB, and SalUn each have configurations where a single step strictly increases retain loss, whereas PURGE prevents this by construction. Combined with the retain-confusion forget target, dual stopping, and GroupNorm, PURGE achieves per-dataset mean FA $\leq 6.3\%$ (excl. STL-10) with RA $> 95\%$ (91% on CIFAR-100) across six datasets and 31 forgetting tasks.

Limitations. PURGE has clear limitations. First, a $\sim$7.5pp test-accuracy gap versus SalUn reflects the cost of conservative, projection-constrained forgetting. Second, STL-10 is a failure case (FA $= 72.8\%$): with only 500 training images per class, the retain set dominates and projection prevents meaningful forgetting. Third, sequential unlearning causes cumulative damage ($\sim$10pp TA drop per step), and fine-tuning on 50 forget-class samples recovers the forgotten class within 5–10 epochs—vulnerabilities shared by all approximate methods. Fourth, the per-step guarantee is exact for the sampled mini-batch retain gradient (what we project against), not the full-population retain gradient; a population-exact per-step guarantee is future work. Finally, PURGE does not provide formal (ε, δ)-DP guarantees.

Broader impact. The CL–MU duality provides a principled and underexploited design space for unlearning algorithm design. PURGE demonstrates that one CL tool—gradient projection—already yields the first per-step retain-safety guarantee for approximate unlearning. Any CL technique that protects old-task knowledge has a natural MU counterpart that protects retained knowledge during erasure. Extending to larger models (ViT), larger datasets (Tiny ImageNet), and establishing a formal DP connection are immediate next steps. Code, configs, logs, and the BN-recalibration diagnostic are publicly released.

References

Bourtoule, L.; Chandrasekaran, V.; Choquette-Choo, C. A.; Jia, H.; Travers, A.; Zhang, B.; Lie, D.; and Papernot, N. 2021. Machine Unlearning. In *IEEE Symposium on Security and Privacy (S&P)*.

Carlini, N.; Chien, S.; Nasr, M.; Song, S.; Terzis, A.; and Tramer, F. 2022. Membership Inference Attacks From First Principles. In *IEEE Symposium on Security and Privacy (S&P)*.

Chaudhry, A.; Ranzato, M.; Rohrbach, M.; and Elhoseiny, M. 2019. Efficient Lifelong Learning with A-GEM. In *International Conference on Learning Representations (ICLR)*.

Chundawat, V. S.; Tarun, A. K.; Mandal, M.; and Kankanhalli, M. 2023. Can Bad Teaching Induce Forgetting? Unlearning in Deep Networks Using an Incompetent Teacher. In *AAAI Conference on Artificial Intelligence*.

Coates, A.; Ng, A.; and Lee, H. 2011. An Analysis of Single-Layer Networks in Unsupervised Feature Learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 215–223.

Fan, C.; Liu, J.; Zhang, Y.; Wei, D.; Wong, E.; and Liu, S. 2024. SalUn: Empowering Machine Unlearning via Gradient-based Weight Saliency. In *International Conference on Learning Representations (ICLR)*.

Foster, J.; Schoepf, S.; and Brintrup, A. 2024. Fast Machine Unlearning without Retraining through Selective Synaptic Dampening. In *AAAI Conference on Artificial Intelligence*, volume 38, 12043–12051.

Golatkar, A.; Achille, A.; Ravichandran, A.; Polito, M.; and Soatto, S. 2021. Forgetting Outside the Class: Scrubbing Deep Networks of Information Available in Training Data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Golatkar, A.; Achille, A.; and Soatto, S. 2020. Eternal Sunshine of the Spotless Net: Selective Forgetting in Deep Networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 9304–9312.

Graves, L.; Nagisetty, V.; and Ganesh, V. 2021. Amnesiac Machine Learning. In *AAAI Conference on Artificial Intelligence*.

Guo, C.; Goldstein, T.; Hannun, A.; and Van Der Maaten, L. 2020. Certified Data Removal from Machine Learning Models. In *International Conference on Machine Learning (ICML)*, 3832–3842.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep Residual Learning for Image Recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

Krizhevsky, A. 2009. Learning Multiple Layers of Features from Tiny Images.

Kurmanji, M.; Triantafillou, P.; Hayes, J.; and Triantafillou, E. 2023. Towards Unbounded Machine Unlearning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

LeCun, Y.; Bottou, L.; Bengio, Y.; and Haffner, P. 1998. Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11): 2278–2324.

Netzer, Y.; Wang, T.; Coates, A.; Bissacco, A.; Wu, B.; and Ng, A. Y. 2011. Reading Digits in Natural Images with Unsupervised Feature Learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*.

Nguyen, Q. P.; Low, B. K. H.; and Jaillet, P. 2020. Variational Bayesian Unlearning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Patel, G.; and Qiu, Q. 2025. Learning to Unlearn while Retaining: Combating Gradient Conflicts in Machine Unlearning. In *IEEE/CVF International Conference on Computer Vision (ICCV)*.

Shokri, R.; Stronati, M.; Song, C.; and Shmatikov, V. 2017. Membership Inference Attacks Against Machine Learning Models. In *IEEE Symposium on Security and Privacy (S&P)*, 3–18.

Thudi, A.; Deza, G.; Chandrasekaran, V.; and Papernot, N. 2022. Unrolling SGD: Understanding Factors of Influence in Machine Unlearning. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, 303–319.

Yang, J.; Shi, R.; Wei, D.; Liu, Z.; Zhao, L.; Ke, B.; Pfister, H.; and Ni, B. 2023. MedMNIST v2 – A Large-Scale Lightweight Benchmark for 2D and 3D Biomedical Image Classification. *Scientific Data*, 10(1): 41.